Mine spørsmål er merkert meg gul.

hvordan brukes df i python til å sette flere ting innu en tabell i sånn df[] osv?

Jeg antar du mener pandas DataFrame. Her er de vanligste måtene å sette inn eller hente ut flere ting med `df[]`:

Velge flere kolonner (bruk dobbel hakeparentes, altså en liste inni):

```
df[["navn", "alder", "by"]]
```

Legge til én ny kolonne:

```
df["ny_kolonne"] = 0                          # samme verdi på alle rader
df["dobbel"] = df["alder"] * 2                # regnet ut fra andre kolonner
df["liste"] = [1, 2, 3, 4]                    # liste med like mange verdier
som rader
```

Legge til flere kolonner samtidig:

```
df[["a", "b"]] = [[1, 2], [3, 4], [5, 6]]    # én rad per rad i df
df[["a", "b"]] = 0                            # samme verdi i begge
```

Eller med `assign`, som er ryddig når du lager flere på en gang:

```
df = df.assign(
    dobbel=df["alder"] * 2,
    over_18=df["alder"] >= 18,
)
```

Sette verdier på bestemte rader (filtrering):

```
df.loc[df["alder"] > 30, "gruppe"] = "eldre"
df.loc[df["alder"] <= 30, "gruppe"] = "yngre"

# flere kolonner på de radene
df.loc[df["alder"] > 30, ["gruppe", "status"]] = ["eldre", "aktiv"]
```

Legge til nye rader:

```
import pandas as pd

ny_rad = pd.DataFrame({"navn": ["Ola"], "alder": [25]})
df = pd.concat([df, ny_rad], ignore_index=True)

# eller én rad direkte med loc
df.loc[len(df)] = ["Kari", 30]
```

Små tips:

- `df["x"]` gir en Series, `df[["x"]]` gir en DataFrame.
- Bruk `.loc[rad, kolonne]` når du setter verdier på utvalg, ikke `df[...][...] = ...`, siden det gir `SettingWithCopyWarning` og kan feile stille.
- Ved lister må lengden matche antall rader, ellers får du `ValueError`.

Hvis du forteller hva du faktisk prøver å gjøre (f.eks. «jeg vil lage en kolonne basert på en betingelse»), kan jeg vise en mer presis løsning.

forklar meg hvordan man lager opusett fra lamda og hvordan man kan bruke den i en kode med apply()

Jeg antar du bruker matplotlib (eller pandas som plotter via matplotlib). Det enkleste er å rotere tickene:

```
import matplotlib.pyplot as plt

plt.xticks(rotation=45)
```

Hvis du vil at teksten skal ende pent på tickene når den er skrå, legg til høyrejustering:

```
plt.xticks(rotation=45, ha="right")
```

Med fig/ax-stil:

```
fig, ax = plt.subplots()
ax.plot(df["dato"], df["temp"])

ax.tick_params(axis="x", labelrotation=45)
# eller
plt.setp(ax.get_xticklabels(), rotation=45, ha="right")
```

Direkte i pandas:

```
df.plot(x="dato", y="temp", rot=45)
```

Hvis den fortsatt ser trang ut eller blir kuttet av:

```
plt.tight_layout()
```

For datoer (vanlig med værdata) finnes det en egen funksjon som roterer og justerer automatisk:

```
fig.autofmt_xdate()
```

Hvis det er veldig mange ticks, kan du vise færre:

```
import matplotlib.dates as mdates

ax.xaxis.set_major_locator(mdates.MonthLocator())              # én tick per
måned
```

```
ax.xaxis.set_major_formatter(mdates.DateFormatter("%b %Y"))   # f.eks. "jan 2025"
```

Eller for vanlige kategorier/tall:

```
plt.xticks(df["x"][::10], rotation=45)   # hver 10. verdi
```

Rotasjon kan være hvilken som helst vinkel, f.eks. `rotation=90` for vertikalt.

Bruker du seaborn, er det samme som matplotlib: `ax = sns.lineplot(...)` og så `ax.tick_params(axis="x", labelrotation=45).`

kan man bruke head() for for å kopierer ikke hele tabellen? og om man skurver det så vil likåvell koden oppfatte alle kolonnene i tillegg til de få?

Ja, `head()` gir bare de første radene, og alle kolonnene følger med. Den begrenser rader, ikke kolonner.

```
df_liten = df.head(10)      # første 10 rader, alle kolonner
df_liten = df.head()        # standard er 5 rader
```

Hvis du lagrer den i en ny variabel, er det en egen tabell med bare de radene, men fortsatt med alle kolonnene. Den originale `df` endres ikke.

Begrense kolonner også:

```
df[["dato", "temp"]].head(10)        # to kolonner, 10 rader
df.head(10)[["dato", "temp"]]        # samme resultat
```

Andre varianter:

```
df.tail(10)          # siste 10 rader
df.iloc[:10, :3]     # første 10 rader og første 3 kolonner
```

Kopi eller ikke?
`df.head(10)` kan oppføre seg som en visning av originalen, så hvis du skal endre i den nye tabellen, ta en ekte kopi for å slippe `SettingWithCopyWarning`:

```
df_liten = df.head(10).copy()
```

Hvis du *skriver ut* eller viser den (`print(df.head())`), ser du bare de første radene, men alle kolonnene, så lenge de får plass. Pandas kan skjule midtkolonner med `...` hvis tabellen er bred. Det er bare visningen som kortes ned, ikke dataene. Vil du se alle kolonner:

```
pd.set_option("display.max_columns", None)
```

hvordan lager man stolpediagram i python

Med matplotlib (eller pandas, som bruker matplotlib):

Enkelt stolpediagram:

```
import matplotlib.pyplot as plt

kategorier = ["Jan", "Feb", "Mar", "Apr"]
verdier = [12, 19, 7, 15]

plt.bar(kategorier, verdier)
plt.xlabel("Måned")
plt.ylabel("Antall")
plt.title("Tittel")
plt.show()
```

Fra en DataFrame:

```
df.plot(x="måned", y="nedbør", kind="bar")
# eller
df.plot.bar(x="måned", y="nedbør", rot=45)
```

Horisontale stolper:

```
plt.barh(kategorier, verdier)
```

Flere serier ved siden av hverandre:

```
df.plot(x="måned", y=["temp", "nedbør"], kind="bar")
```

Med ren matplotlib må du forskyve stolpene selv:

```
import numpy as np

x = np.arange(len(kategorier))
bredde = 0.4

plt.bar(x - bredde/2, serie1, bredde, label="Serie 1")
plt.bar(x + bredde/2, serie2, bredde, label="Serie 2")
plt.xticks(x, kategorier)
plt.legend()
```

Stablede stolper:

```
df.plot(x="måned", y=["a", "b"], kind="bar", stacked=True)
```

Telle forekomster og plotte dem (veldig vanlig):

```
df["kategori"].value_counts().plot(kind="bar")
```

Gruppere og plotte gjennomsnitt:

```
df.groupby("måned")["temp"].mean().plot(kind="bar")
```

Pynt:

```
plt.bar(kategorier, verdier, color="steelblue", edgecolor="black")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
```

Med seaborn:

```
import seaborn as sns

sns.barplot(data=df, x="måned", y="nedbør")
```

Merk at `sns.barplot` viser gjennomsnitt per kategori som standard, og tegner feilstolper. Vil du heller telle antall rader, bruk `sns.countplot(data=df, x="kategori")`.

Til forskjell fra stolpediagram brukes `plt.hist(df["temp"], bins=20)` for histogram, altså fordeling av en tallserie.

så det er bar() som er stolpediagrammet?

Ja, `bar()` er stolpediagrammet i matplotlib:

```
plt.bar(kategorier, verdier)      # vertikale stolper
plt.barh(kategorier, verdier)     # horisontale stolper
```

I pandas kan du skrive det på to måter, med samme resultat:

```
df.plot(kind="bar")
df.plot.bar()
```

Merk forskjellen på `bar()` og `hist()`: `bar()` viser verdier per kategori (en stolpe per måned, by osv.), mens `hist()` deler en tallserie inn i intervaller og viser hvor mange verdier som havner i hvert (fordeling).

hvordan gjør jeg det mindre synelig?

Mest sannsynlig mener du gjennomsiktighet, som styres med `alpha` (0 = usynlig, 1 = full farge):

```
plt.bar(kategorier, verdier, alpha=0.5)
df.plot(x="måned", y="nedbør", kind="bar", alpha=0.5)
```

Det er spesielt nyttig når du tegner flere ting oppå hverandre og vil se gjennom:

```
plt.bar(kategorier, serie1, alpha=0.5, label="Serie 1")
plt.bar(kategorier, serie2, alpha=0.5, label="Serie 2")
plt.legend()
```

Andre måter å gjøre det mindre fremtredende på:

Lysere eller grå farge:

```
plt.bar(kategorier, verdier, color="lightgray")
```

Smalere stolper (standard bredde er 0.8):

```
plt.bar(kategorier, verdier, width=0.5)
```

Fjerne kanten:

```
plt.bar(kategorier, verdier, edgecolor="none")
```

Svakere rutenett eller ingen:

```
plt.grid(alpha=0.3)     # svakt rutenett
plt.grid(False)         # ingen rutenett
```

Skjule rammen rundt plottet:

```
ax = plt.gca()
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
```

Hvis du mente noe annet, f.eks. å skjule hele plottet, bare akser eller tall på aksene, si fra, så viser jeg det.

hvordan var det man plotta inn to ulike verdier på y aksen, Assa to ting på to takser typ i grafen

Du tenker på to y-akser, og det gjør du med `twinx()`. Den lager en ekstra y-akse til høyre som deler samme x-akse.

```
import matplotlib.pyplot as plt

fig, ax1 = plt.subplots()

ax1.plot(df["dato"], df["temp"], color="tomato")
ax1.set_ylabel("Temperatur (°C)", color="tomato")

ax2 = ax1.twinx()                       # ny y-akse til høyre
ax2.bar(df["dato"], df["nedbør"], color="steelblue", alpha=0.5)
ax2.set_ylabel("Nedbør (mm)", color="steelblue")

fig.autofmt_xdate()                     # roterer datoene
plt.show()
```

Her blir temperatur på venstre akse og nedbør på høyre.

Direkte i pandas:

```
ax = df.plot(x="dato", y="temp", color="tomato", legend=False)
df.plot(x="dato", y="nedbør", kind="line", secondary_y=True, ax=ax)
```

Eller enda enklere:

```
df.plot(x="dato", y=["temp", "nedbør"], secondary_y="nedbør")
```

Farge på tallene på aksen også:

```
ax1.tick_params(axis="y", colors="tomato")
ax2.tick_params(axis="y", colors="steelblue")
```

Samlet forklaringsboks (legend): Med `twinx` får hver akse sin egen legend. Vil du ha én samlet:

```
l1, n1 = ax1.get_legend_handles_labels()
l2, n2 = ax2.get_legend_handles_labels()
ax1.legend(l1 + l2, n1 + n2, loc="upper left")
```

Husk å sette `label="..."` på hver plot-kommando for at dette skal fungere.

Tips:

- Når du blander linje og stolper, kan stolpene havne oppå linjen. Tegn stolpene på `ax1` og linjen på `ax2`, eller bruk `ax1.set_zorder(ax2.get_zorder() + 1)` og `ax1.patch.set_visible(False)` for å styre rekkefølgen.
- To y-akser kan lett gi misvisende bilder, siden skalaene er uavhengige. Merk aksene tydelig.

for å få grid i python så bruker jeg grid() men hva må være inni parentesen?

Ingenting må stå inni. `plt.grid()` alene slår på rutenettet (og `plt.grid()` en gang til slår det av igjen, siden den toggler hvis du ikke oppgir noe). Tryggest er å være eksplisitt:

```
plt.grid(True)      # på
plt.grid(False)     # av
```

Vanlige valg inni parentesen:

```
plt.grid(True, alpha=0.3)            # svakt rutenett
plt.grid(True, linestyle="--")       # stiplet
plt.grid(True, color="gray", linewidth=0.5)
```

Bare én akse:

```
plt.grid(True, axis="y")     # bare horisontale linjer
plt.grid(True, axis="x")     # bare vertikale linjer
```

Hovedlinjer eller også de små (minor):

```
plt.grid(True, which="major")   # standard
plt.grid(True, which="minor")   # krever at minor ticks er slått på
plt.grid(True, which="both")
```

Med fig/ax-stil:

```
ax.grid(True, axis="y", alpha=0.3)
```

Med to y-akser (`twinx`): Sett grid på bare én av aksene, ellers får du to rutenett oppå hverandre som ikke stemmer overens:

```
ax1.grid(True, alpha=0.3)
ax2.grid(False)
```

Pandas: `df.plot(grid=True)` fungerer også.

Tips: Vil du at rutenettet skal ligge bak stolpene/linjene, bruk `ax.set_axisbelow(True)`.

forklar meg i dybden yaml biblioteket

YAML er et dataformat for konfigurasjon og strukturert data, laget for å være lett å lese for mennesker. I Python bruker man vanligvis biblioteket **PyYAML**, som importeres som `yaml`.

```
pip install pyyaml
import yaml
```

YAML-syntaksen

```
stasjon: Ås
id: SN17850
aktiv: true
høyde: 94.0
kommentar: null

målinger:
  - dato: 2025-07-01
    temp: 28.4
  - dato: 2025-07-02
    temp: 30.1

tags: [sommer, varme]
```

- Struktur lages med **innrykk** (mellomrom, aldri tab).
- `nøkkel: verdi` lager en dictionary.
- `- element` lager en liste.
- `[a, b]` og `{a: 1, b: 2}` er kortformer for liste og dict.
- `#` starter en kommentar.
- Typer gjenkjennes automatisk: `true/false` blir bool, `null` blir `None`, tall blir int/float, og `2025-07-01` blir `datetime.date`.

Lese YAML (parse)

Fra fil:

```
with open("data.yml", encoding="utf-8") as f:
    data = yaml.safe_load(f)

print(data["stasjon"])                  # Ås
print(data["målinger"][0]["temp"])      # 28.4
```

Fra tekststreng:

```
data = yaml.safe_load("a: 1\nb: [2, 3]")
# {'a': 1, 'b': [2, 3]}
```

Resultatet er vanlige Python-objekter (dict, list, str, int, float, bool, None, date). Du jobber med dem som alt annet i Python.

Bruk alltid `safe_load`, ikke `load`. Den vanlige `yaml.load` kan (avhengig av loader) lage vilkårlige Python-objekter og dermed kjøre kode fra en ondsinnet fil. `safe_load` støtter bare grunnleggende typer.

Skrive YAML (dump)

```
data = {
    "stasjon": "Ås",
    "id": "SN17850",
    "heatwaves": [
        {"start": "2025-07-01", "slutt": "2025-07-04", "maks": 31.2},
        {"start": "2025-07-20", "slutt": "2025-07-23", "maks": 29.8},
    ],
}

with open("ut.yml", "w", encoding="utf-8") as f:
    yaml.safe_dump(data, f, sort_keys=False, allow_unicode=True,
default_flow_style=False)
```

Viktige argumenter:

Argument	Hva det gjør
`sort_keys=False`	Beholder rekkefølgen din (standard sorterer alfabetisk)
`allow_unicode=True`	Beholder æ, ø, å i stedet for `\xe5`-koder
`default_flow_style=False`	Bruker blokkstil (innrykk) i stedet for `{a: 1}`-stil
`indent=4`	Antall mellomrom per innrykk
`width=...`	Maks linjelengde før den bryter linjer

Uten filobjekt får du en streng:

```
tekst = yaml.safe_dump(data, sort_keys=False, allow_unicode=True)
print(tekst)
```

Flere dokumenter i én fil

Et dokument skilles med `---`:

```
---
navn: a
---
navn: b
with open("flere.yml") as f:
    for dok in yaml.safe_load_all(f):
        print(dok)

yaml.safe_dump_all([dict1, dict2], f)
```

Ankre og aliaser (gjenbruk)

```
standard: &std
  enhet: celsius
  kilde: MET

sensor1:
  <<: *std
  id: 1
```

`&std` navngir en blokk, `*std` gjenbruker den, og `<<:` slår den sammen inn i en annen dict.

Flerlinjet tekst

```
beskrivelse: |
  Linje 1
  Linje 2        # | beholder linjeskift

sammendrag: >
  Dette blir
  én lang linje  # > slår sammen linjene
```

Typiske fallgruver

1. Tall og tekst blandes. `versjon: 1.10` blir flyttallet `1.1`. Skriv `versjon: "1.10"` hvis det skal være tekst.

2. «Norge-problemet». I YAML 1.1 (som PyYAML følger)
tolkes `NO`, `no`, `off` og `n` som `False`, og `yes`, `on` som `True`. Landskoden for Norge forsvinner dermed:

```
yaml.safe_load("land: NO")    # {'land': False}
```

Løsning: bruk anførselstegn, `land: "NO"`.

3. Datoer blir date-objekter. `dato: 2025-07-01` gir `datetime.date`, ikke streng. Ønsker du tekst, bruk `"2025-07-01"` eller `str(...)` når du leser.

4. Kolon i tekst. `tittel: Rapport: del 1` feiler. Bruk `tittel: "Rapport: del 1"`.

5. Tab i stedet for mellomrom gir `ScannerError`.

6. `dump` av spesialobjekter. `safe_dump` klarer ikke numpy-tall, pandas-objekter eller `Timestamp`. Konverter først:

```
data = {"maks": float(df["temp"].max()), "dato": str(df["dato"].iloc[0])}
```

Fra en DataFrame:

```
data = {"målinger": df.to_dict(orient="records")}
```

(husk å gjøre datoer og numpy-typer om til vanlige typer før `safe_dump`).

Feilhåndtering

```
try:
    with open("data.yml", encoding="utf-8") as f:
        data = yaml.safe_load(f)
except FileNotFoundError:
    print("Fant ikke filen")
except yaml.YAMLError as e:
    print("Ugyldig YAML:", e)       # sier ofte linje og kolonne
```

Egne klasser (avansert)

Med `yaml.dump` (ikke `safe_dump`) kan Python-objekter serialiseres med `!!python/object`-tagger, men da må du bruke `yaml.load(..., Loader=yaml.FullLoader)` eller verre ved innlesing, noe som er usikkert. Bedre er å gjøre objektet om til en dict før du skriver, og bygge det opp igjen etter lesing. Vil du likevel ha egne typer på en trygg måte, kan du registrere en representer/konstruktør:

```
def dato_rep(dumper, d):
    return dumper.represent_scalar("!dato", d.isoformat())

yaml.add_representer(MinKlasse, dato_rep, Dumper=yaml.SafeDumper)
```

Alternativer

- **ruamel.yaml** støtter YAML 1.2 (løser Norge-problemet), og beholder kommentarer og rekkefølge når du leser og skriver tilbake. Bra når du redigerer eksisterende filer.
- **JSON** (`import json`) er enklere og strengere, men uten kommentarer og mindre lesbart.
- **TOML** (`tomllib`) er vanlig for konfigurasjon.

Oppsummering

```
import yaml
```

```
# lese
with open("in.yml", encoding="utf-8") as f:
    data = yaml.safe_load(f)

# skrive
with open("ut.yml", "w", encoding="utf-8") as f:
    yaml.safe_dump(data, f, sort_keys=False, allow_unicode=True)
```

Husk `safe_load`/`safe_dump`, anførselstegn rundt tekst som kan misforstås (`"NO"`, `"1.10"`), og konverter numpy/pandas-typer til vanlige Python-typer før du skriver. Vil du ha et konkret eksempel, f.eks. å skrive resultater fra en DataFrame til YAML, kan jeg vise det.

men nåt må jeg bruke yaml

Da er det i praksis to ting: skrive en dict til en `.yml`-fil, og eventuelt lese en inn. Ut fra eksempelfilen din (`heatwave_summary_example.yml`) ser det ut til at du skal ha nøklene `dates`, `station_id`, `station_name` og `year`. Da blir det slik:

```
import yaml

resultat = {
    "station_id": "SN17850",
    "station_name": "Ås",
    "year": 2025,
    "dates": ["2025-07-01", "2025-07-02", "2025-07-03"],
}

with open("heatwave_summary.yml", "w", encoding="utf-8") as f:
    yaml.safe_dump(resultat, f, sort_keys=False, allow_unicode=True)
```

Husk:

- `sort_keys=False` beholder rekkefølgen din (ellers sorteres nøklene alfabetisk).
- `allow_unicode=True` gjør at «Ås» ikke blir `\xc5s`.
- Datoer må være vanlige strenger (eller `datetime.date`), ikke pandas `Timestamp`, ellers feiler `safe_dump`. Fra en kolonne:

```
dates = df.loc[df["heatwave"], "dato"].dt.strftime("%Y-%m-%d").tolist()
```

- Tall fra pandas/numpy må gjøres om til vanlige typer, f.eks. `int(...)` eller `float(...)`.

Lese en yml-fil (for å se hvordan eksempelet er bygd opp):

```
with open("heatwave_summary_example.yml", encoding="utf-8") as f:
    eksempel = yaml.safe_load(f)
print(eksempel)
```

Skriv ut `eksempel` og sammenlign med din egen dict, så ser du om strukturen (f.eks. om `dates` er en liste eller en dict med start/slutt) er lik.

Hvis du limer inn innholdet i eksempelfilen, kan jeg vise nøyaktig hvordan du bygger dicten din slik at den matcher.

forklar meg i dybden loc, jeg forsto ikke fra nettet så si det enklere

`loc` er en måte å si **«gi meg disse radene og disse kolonnene»** i en tabell. Det er alltid to deler:

```
df.loc[hvilke_rader, hvilke_kolonner]
```

Rader først, kolonner etterpå, atskilt med komma. Tenk på det som Excel: først velger du rader, så kolonner.

Eksempeltabell (jeg bruker denne videre):

```
import pandas as pd

df = pd.DataFrame({
    "navn": ["Ola", "Kari", "Per", "Lise"],
    "alder": [25, 32, 41, 19],
    "by": ["Oslo", "Ås", "Oslo", "Bergen"],
}, index=["a", "b", "c", "d"])
  navn  alder     by
a  Ola     25   Oslo
b Kari     32     Ås
c  Per     41   Oslo
d Lise     19 Bergen
```

Legg merke til radene har navn (`a`, `b`, `c`, `d`). Det kalles **indeksen**. `loc` bruker *navn*, ikke posisjon.

1. Én celle

```
df.loc["b", "alder"]     # 32
```

Rad `b`, kolonne `alder`.

2. Én hel rad

```
df.loc["b"]              # hele raden til Kari
```

3. Flere rader eller kolonner (bruk liste)

```
df.loc[["a", "c"], "navn"]              # Ola og Per, bare navn
df.loc[["a", "c"], ["navn", "by"]]      # to rader, to kolonner
```

4. Et område (slice)

```
df.loc["a":"c", "navn":"alder"]
```

Viktig: med `loc` er **slutten med**. `"a":"c"` gir a, b og c. (Det er annerledes enn vanlig Python og `iloc`.)

5. Alle rader / alle kolonner: `:`

```
df.loc[:, "alder"]       # alle rader, kolonnen alder
df.loc["b", :]           # raden b, alle kolonner
```

Kolonnen `:` betyr «alle».

6. Filtrering med betingelse (det du bruker mest)

Her er det en sann/usann-liste som velger rader:

```
df["alder"] > 30
```

gir

```
a    False
b     True
c     True
d    False
```

Setter du den inn i `loc`, får du bare radene med `True`:

```
df.loc[df["alder"] > 30]                 # Kari og Per
df.loc[df["alder"] > 30, "navn"]         # bare navnene deres
df.loc[df["alder"] > 30, ["navn", "by"]] # navn og by
```

Flere betingelser: bruk `&` (og), `|` (eller), og **parenteser rundt hver**:

```
df.loc[(df["alder"] > 20) & (df["by"] == "Oslo")]
df.loc[(df["by"] == "Ås") | (df["by"] == "Bergen")]
```

Du kan **ikke** bruke `and`/`or` her, det gir feil.

7. Sette verdier (andre bruken)

Samme `loc`, men med `=` etterpå:

```
df.loc["a", "alder"] = 26                        # endre én celle
df.loc[df["alder"] > 30, "gruppe"] = "eldre"     # ny kolonne, bare på de
radene
df.loc[df["by"] == "Oslo", "alder"] += 1         # legg til 1 for alle i
Oslo
```

Den siste linjen i midten leser du slik: «på radene der alder er over 30, sett kolonnen `gruppe` til `eldre`». Radene som ikke passer får `NaN` i den nye kolonnen.

Dette er riktig måte å sette verdier på. Skriver du `df[df["alder"] > 30]["gruppe"] = "eldre"` (to hakeparenteser etter hverandre), kan det stille feile, og du får `SettingWithCopyWarning`.

loc vs iloc

	Bruker	Eksempel
`loc`	**navn** (indeksnavn og kolonnenavn)	`df.loc["b", "alder"]`
`iloc`	**posisjon** (0, 1, 2, ...)	`df.iloc[1, 1]`

```
df.iloc[0]          # første rad
df.iloc[:2, :2]     # to første rader, to første kolonner (slutten er IKKE med)
```

Husk: **L**oc = **L**abel (navn), **I**loc = **I**nteger (tall).

Vanlig forvirring: indeksen er ofte bare tall

Hvis du ikke har gitt radene navn, er navnene 0, 1, 2, ... og da ser `loc` og `iloc` like ut:

```
df2 = df.reset_index(drop=True)
df2.loc[0, "navn"]      # fungerer, 0 er et *navn* her
```

Men etter at du har filtrert, kan navnene hoppe (f.eks. 1, 2 og 5), og da peker `loc[0]` på ingenting, mens `iloc[0]` alltid er første rad i det du har.

Eksempel med datoer (værdata)

```
# alle dager med temp over 25
df.loc[df["temp"] > 25]

# bare dato og temp for de dagene
df.loc[df["temp"] > 25, ["dato", "temp"]]

# marker varme dager
df.loc[df["temp"] > 25, "varm"] = True
df.loc[df["temp"] <= 25, "varm"] = False
```

Kort oppsummert

```
df.loc[rader, kolonner]
```

- `rader`: navn, liste av navn, område `"a":"c"`, `:` for alle, eller en sann/usann-betingelse.
- `kolonner`: samme, men for kolonner.
- Bruk det til å **hente** (`x = df.loc[...]`) og til å **sette** (`df.loc[...] = verdi`).

Vil du øve, kan du skrive hva du prøver å hente ut av din egen tabell, så skriver jeg `loc`-linjen for deg.

hvordan leger jeg punktdiagram i grafen

Punktdiagram (scatter plot) lager du med `scatter()`:

```
import matplotlib.pyplot as plt

plt.scatter(df["dato"], df["temp"])
plt.xlabel("Dato")
plt.ylabel("Temperatur (°C)")
plt.show()
```

Direkte i pandas:

```
df.plot.scatter(x="dato", y="temp")
# eller
df.plot(x="dato", y="temp", kind="scatter")
```

Pynt:

```
plt.scatter(df["dato"], df["temp"],
            color="tomato",     # farge
            s=20,               # størrelse på punktene
            alpha=0.6,          # gjennomsiktighet
            marker="o",         # form: "o", "x", "^", "s", "*"
            label="Temp")
plt.legend()
```

Farge etter verdi (f.eks. varmere dager blir rødere):

```
plt.scatter(df["dato"], df["temp"], c=df["temp"], cmap="coolwarm")
plt.colorbar(label="°C")
```

Fremheve bestemte punkter (f.eks. hetebølgedager) oppå resten:

```
plt.scatter(df["dato"], df["temp"], color="lightgray", s=10)

varme = df.loc[df["temp"] > 25]
plt.scatter(varme["dato"], varme["temp"], color="red", s=20, label="Over 25
°C")
plt.legend()
```

Bare punkter på en vanlig linje (ikke scatter, men plot med markører):

```
plt.plot(df["dato"], df["temp"], marker="o")             # linje + punkter
plt.plot(df["dato"], df["temp"], "o")                    # bare punkter
plt.plot(df["dato"], df["temp"], marker="o", linestyle="none")
```

Med datoer på x-aksen, husk å rotere dem:

```
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
```

Med seaborn:

```
import seaborn as sns
sns.scatterplot(data=df, x="dato", y="temp", hue="by")
```

`hue` gir ulik farge per kategori.

Forskjellen: `scatter()` er best når hvert punkt kan ha egen farge eller størrelse. `plot(..., "o")` er raskere hvis du har veldig mange punkter og alle skal se like ut.

er det lurt å bruke thight layout?

Ja, som regel. `plt.tight_layout()` justerer marginene automatisk slik at aksetitler, tall på aksene og tittel ikke blir kuttet av eller overlapper. Det er spesielt nyttig når du har roterte x-etiketter, lange titler eller flere delplott.

```
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

Regler:

- Kall den **etter** at du har tegnet alt (plott, titler, etiketter, rotasjon), men **før** `plt.show()` eller `plt.savefig()`.
- Med `fig/ax`-stil: `fig.tight_layout()`.
- Når du lagrer bildet, kan du også bruke `plt.savefig("fig.png", bbox_inches="tight")`, som beskjærer unødvendig hvit kant og hindrer at ting blir kuttet.

Når den ikke hjelper eller kan skape trøbbel:

- Ved veldig lange etiketter kan plottet bli krympet til nesten ingenting. Da hjelper det å korte ned teksten, øke figurstørrelsen (`plt.subplots(figsize=(10, 5))`) eller vise færre ticks.
- Med `twinx()` og legend plassert utenfor plottet kan den noen ganger ikke ta hensyn til alt.
- Den gir av og til en advarsel («not compatible with all Axes»), da kan du prøve `constrained_layout`:

```
fig, ax = plt.subplots(constrained_layout=True)
```

`constrained_layout` er en nyere og ofte bedre variant, men `tight_layout()` er enklest og fungerer i de fleste tilfeller.